

# TSDF\_seqKANSeq (KAN-only)

## Resultados e Estrutura do Treino

Fernanda Mickosz Villa Verde

CPE-883 - Aprendizado de Máquina

1 de março de 2026

# Sumário

1 Resumo

2 Metodologia

3 Resultados

# Resumo (compatível com o código atual)

- Modelo: **TSDF\_seqKANSeq** em modo **KAN-only**: sem MLP decoder, sem VAE, sem miss-head, sem status/static.
- Entrada efetiva do backbone (por timestep): **concatenação explícita**

$$s_t = [x_t, m_t] \in \mathbb{R}^{2C}$$

onde  $m_t$  é máscara feature-wise (1=observado, 0=ausente). No código:  $h = \text{cat}([x, \text{mask}], -1)$ .

- Célula recorrente: **SeqKANSeq** computa

$$h_t = \text{KAN}_{\text{cell}}([s_t, h_{t-1}]), \quad \hat{x}_t = \text{KAN}_{\text{out}}(h_t)$$

- Treino supervisionado por reconstrução (MSE). O bloco de difusão injeta ruído em **[x,mask]** (opcionalmente ruído também na máscara).

# KAN: ideia central

- Em redes usuais (MLP), cada neurônio combina entradas por pesos lineares:

$$y_j = \sum_i w_{ij} z_i$$

- No **KAN**, cada conexão  $i \rightarrow j$  é uma **função univariada aprendida**:

$$y_j = \sum_i f_{ij}(z_i)$$

- Consequências: estrutura **aditiva**, maior **interpretabilidade por conexão** e controle direto do grafo.

## Intuição

*Troca “peso” por “função 1D” em cada aresta.*

# KAN\_cell e KAN\_out: o que são esses dois blocos?

- O SeqKANSeq é composto por dois módulos KAN com papéis diferentes:
  - **KAN\_cell (transição do estado)**: atualiza a memória do modelo.
  - **KAN\_out (leitura/saída)**: transforma a memória em saída/reconstrução.
- **KAN\_cell** implementa a dinâmica recorrente:

$$h_t = \text{KAN}_{\text{cell}}([s_t, h_{t-1}]), \quad s_t = [x_t, m_t]$$

Intuição: “como a entrada atual e o estado anterior se combinam para formar o novo estado”.

- **KAN\_out** implementa o head de reconstrução:

$$\hat{x}_t = \text{KAN}_{\text{out}}(h_t)$$

Intuição: “como ler o estado oculto para reconstruir as features”.

- Ambos são **KAN**: em vez de pesos lineares, usam somas de **funções univariadas por conexão**, o que facilita controle estrutural (ex.: Top-k) e interpretabilidade.
- Diferença para GRU/LSTM: não há gates multiplicativos; a não linearidade está nas **funções das conexões**.

# O que são as splines no KAN

- No KAN, cada conexão  $i \rightarrow j$  aprende uma função univariada  $f_{ij}(\cdot)$ . Em vez de escolher uma ativação fixa (ReLU, tanh), o modelo **aprende o formato da curva**.
- Essa função é parametrizada como uma **spline**: uma curva **por partes** definida em intervalos do eixo- $x$ .
- O domínio é dividido em **nós** (grid). Em cada intervalo entre nós, a spline usa um polinômio simples, mas a junção entre intervalos é feita de forma **suave** (continuidade).
- Intuição prática:
  - mais nós  $\Rightarrow$  mais flexibilidade para capturar não linearidades;
  - menos nós  $\Rightarrow$  função mais simples e mais estável.
- Interpretabilidade: ao inspecionar a curva  $f_{ij}(u)$ , vemos **como uma entrada específica influencia** o neurônio de saída.

# Splines no KAN: função aprendida por conexão ( $k=3$ )

- No KAN, cada conexão  $i \rightarrow j$  aprende uma função univariada

$$f_{ij} : \mathbb{R} \rightarrow \mathbb{R}, \quad u \mapsto f_{ij}(u)$$

- Em vez de uma ativação fixa (ReLU/tanh),  $f_{ij}$  é parametrizada como uma **B-spline**:

$$f_{ij}(u) = \sum_{m=1}^M \alpha_{ijm} B_m^{(3)}(u)$$

- Interpretação dos termos:
  - $B_m^{(3)}(u)$ : bases B-spline de **ordem 3** (grau 2, quadráticas),
  - $\alpha_{ijm}$ : coeficientes aprendidos por treino,
  - $M$ : número de bases (depende do número de nós no grid).
- Intuição: cada aresta aprende uma **curva suave**  $u \mapsto f_{ij}(u)$ , inspecionável e controlável.

# Nós (grid), suporte local e suavidade ( $k=3$ )

- A spline é definida a partir de uma sequência de nós (grid)

$$t_0 \leq t_1 \leq \dots \leq t_K, \quad u \in [t_0, t_K]$$

- Suporte local:** cada base afeta apenas uma região do eixo- $u$ :

$$B_m^{(3)}(u) = 0 \quad \text{fora de } [t_m, t_{m+3}]$$

- Suavidade:** com  $k = 3$  (grau 2), a função aprendida é tipicamente

$$f_{ij} \in C^1$$

isto é,  $f_{ij}$  e sua primeira derivada são contínuas (exceto se houver nós repetidos).

- Trade-off do grid:
  - mais nós  $\Rightarrow$  maior flexibilidade para capturar não linearidades finas;
  - menos nós  $\Rightarrow$  função mais simples e mais estável.

## Conexão com o SeqKANSeq

Essas splines parametrizam as conexões tanto na  $\text{KAN}_{\text{cell}}$  (dinâmica) quanto na  $\text{KAN}_{\text{out}}$  (reconstrução).



# Esparsidade Estrutural (Top-k) por neurônio

- Para aumentar **interpretabilidade** e **controle estrutural**, aplicamos **Top-k** nas conexões do KAN.
- Como o KAN é aditivo por arestas, cada neurônio  $j$  recebe contribuições por conexões  $i \rightarrow j$ :

$$y_j = \sum_i f_{ij}(z_i)$$

- Com Top-k **por neurônio**, mantemos apenas as  $k$  conexões mais relevantes **para cada neurônio**  $j$ :

$$y_j = \sum_{i \in \mathcal{S}_j} f_{ij}(z_i), \quad |\mathcal{S}_j| \leq k$$

- Interpretação: cada neurônio passa a depender de um **conjunto pequeno e estável de entradas**, o que simplifica a leitura do grafo e reduz conexões fracas/ruidosas.

## Onde aplicamos

Na célula recorrente ( $[x_t, m_t]$  e  $h_{t-1}$ ) e/ou no head de saída ( $h_t \rightarrow \hat{x}_t$ ).

# Top-k na célula recorrente (entrada vs memória)

- Na atualização do estado, o SeqKANSeq combina **entrada** e **memória**:

$$h_t = \text{KAN}_{cell}([x_t, m_t, h_{t-1}])$$

- Com **Top-k por neurônio**, cada neurônio oculto seleciona apenas um número limitado de conexões relevantes.
- Essa seleção pode ser feita **separando as fontes**:
  - até  $k_x$  conexões vindas da **entrada** ( $[x_t, m_t]$ ),
  - até  $k_h$  conexões vindas da **memória** ( $h_{t-1}$ ),
  - (opcional) tratar a **máscara**  $m_t$  separadamente para não competir com as features de  $x_t$ .
- Intuição: controlar **quantas variáveis do instante** e **quanta memória passada** cada neurônio pode usar.

# Top-k no head de saída (reconstrução)

- A reconstrução é produzida por um head (KAN) que lê o estado:

$$\hat{x}_t = \text{KAN}_{out}(h_t)$$

- Com **Top-k por saída**, cada componente  $\hat{x}_{t,k}$  depende de apenas algumas dimensões de  $h_t$ : isso limita quantos estados ocultos influenciam cada feature reconstruída.
- Em todos os casos, o Top-k atua como **máscara estrutural**: ele **remove conexões** sem mudar a forma das funções nas conexões mantidas.
- A seleção respeita as conexões permitidas pelo modelo-base (máscara estrutural inicial).

# Resultados

| Modelo                                                                           | Loss (train) | Teste MSE ( $\mu \pm SE$ ) |
|----------------------------------------------------------------------------------|--------------|----------------------------|
| TSDF_GRU (hidden 8)                                                              | 0.189331     | 0.147292 $\pm$ 0.003195    |
| TSDF_GRU (hidden 24)                                                             | 0.181741     | 0.145757 $\pm$ 0.003234    |
| TSDF_GRU (hidden 11 * 32)                                                        | 0.184935     | 0.145667 $\pm$ 0.003229    |
| TSDF_SeqKANSeq (grid 8 / hidden 8 / $k_x=4$ $k_m=2$ $k_h=4$ $k_{out}=4$ )        | 0.250516     | 0.175399 $\pm$ 0.003855    |
| TSDF_SeqKANSeq (grid 8 / hidden 8 / $k_x=2$ $k_m=3$ $k_h=2$ $k_{out}=2$ )        | 0.220477     | 0.163581 $\pm$ 0.003494    |
| TSDF_SeqKANSeq (grid 8 / hidden 24 / $k_x=4$ $k_m=2$ $k_h=6$ $k_{out}=8$ )       |              |                            |
| TSDF_SeqKANSeq (grid 8 / hidden 11*32 / $k_x=4$ $k_m=2$ $k_h=88$ $k_{out}=117$ ) | (preencher)  | (preencher)                |

# Comparação Final por Grupos – Test Results

| Modelo                                                                           | MSE Grupo 0 ( $\mu \pm SE$ ) | MSE Grupo 1 ( $\mu \pm SE$ ) |
|----------------------------------------------------------------------------------|------------------------------|------------------------------|
| TSDF_GRU (hidden 8)                                                              | (preencher)                  | (preencher)                  |
| TSDF_GRU (hidden 24)                                                             | (preencher)                  | (preencher)                  |
| TSDF_GRU (hidden 11 * 32)                                                        | (preencher)                  | (preencher)                  |
| TSDF_SeqKANSeq (grid 8 / hidden 8 / $k_x=4$ $k_m=2$ $k_h=4$ $k_{out}=4$ )        | (preencher)                  | (preencher)                  |
| TSDF_SeqKANSeq (grid 8 / hidden 24 / $k_x=4$ $k_m=2$ $k_h=6$ $k_{out}=8$ )       | (preencher)                  | (preencher)                  |
| TSDF_SeqKANSeq (grid 8 / hidden 11*32 / $k_x=4$ $k_m=2$ $k_h=88$ $k_{out}=117$ ) | (preencher)                  | (preencher)                  |

# Conclusão (alinhada ao código)

O TSDF\_seqKANSeq (KAN-only) implementa:

$$h_t = F([x_t, m_t], h_{t-1}), \quad \hat{x}_t = G(h_t)$$

com:

- Não linearidade distribuída nas arestas (B-splines do ArticleKAN)
- Recorrência explícita sem gates multiplicativos (estilo GRU/LSTM)
- Top-k estrutural opcional aplicado à **máscara de conexões** após warmup
- Separação explícita do top-k na célula:  $x$  vs  $m$  vs  $h_{t-1}$  (quando configurado)

Resultados mostram gap em relação ao GRU nas configs atuais, mas com maior controle estrutural e interpretabilidade por aresta.